

Imitation Learning as an Alternative to Reinforcement Learning for Human-AI Cooperation

Theodore Schwartz
Briarcliff High School
Briarcliff Manor, NY

Acknowledgements

I would like to thank Mrs. Annmarie O'Brien for her continued help and support as my science research teacher. She guided me through the process of doing research, from securing a mentor to writing my paper, taught me time management, presentation skills, reading skills, and much more. I would also like to thank Mr. Christopher Lo for his help with coding. He took time to help me understand errors and debugged with me, and taught me Python before the project began. I would also like to thank Mr. Edmund Tsou for his continued help as a peer. He helped me with finding documentation and understanding methods I did not understand, and assisted with concepts in AI I did not understand. I would also like to thank Mr. Tristan Rutman, Mr. Johnathan Iwai, and Mr. Rian Kelmendi for their help as peers in the program. Lastly, I would like to thank Mr. Rujikorn Charakorn for his extremely generous support, offering me his codebase, helping me debug, and answering all my questions. He truly exceeded my expectations of what a mentor was, and he was incredibly important in this project.

Table of Contents

<i>Abstract:</i>	5
1. Introduction	6
1.1 Reinforcement Learning Overview	7
1.2 Imitation Learning Overview	9
1.3 Learning Incompatible Strategies (LIPO) Overview	9
1.4 Environments	10
1.4.1 The Hanabi Learning Environment	10
1.4.2 The Overcooked Environments	11
1.5 Statement of Purpose	14
2. Methods and Materials	14
2.1 Creation of the Training Method	15
2.1.1 Collection of Offline Data from Specialist Agents	16
2.1.2 Training an Agent to Mimic Specialists	16
2.2 Training an Agent to Mimic LIPO-generated Training Partners	17
2.2.1 Collecting Offline Data from LIPO-Generated Partners	17
2.2.2 Training an Agent to Mimic LIPO-Generated Partners	17
2.2.3 Collecting Offline Data from the LIPO-Trained Agent	17
3. Results and Discussion	17
3.1 Results	17
3.2 Discussion	18
3.2.1 High Standard Deviations for Imitation-Lettuce, Imitation-Onion, and Imitation-Tomato Lettuce Teams	18
3.2.2 Extremely Low Relative Performance with Imitation-Lettuce and Imitation-Tomato Lettuce Teams	20
3.2.3 Overall Lower Performance of Imitation Agent vs. RL Agent	20
4. Conclusions	21
5. Future Work	21
6. References	22

List of Figures and Tables

<i>Figure 1. The Reinforcement Learning Cycle, (What is Reinforcement, n.d.)</i>	<i>8</i>
<i>Figure 2, The Overcooked Environment (Carroll et al.)</i>	<i>11</i>
<i>Figure 3, Experimental Overcooked Layouts (Carroll et al.)</i>	<i>12</i>
<i>Figure 4, Overview of the Multi-Recipe Overcooked Environment (Charakorn et al., 2023)</i>	<i>13</i>
<i>Figure 5, Overview of the Steps Taken to Create and Debug the Training Method</i>	<i>15</i>
<i>Figure 6, Overview of the Training and Testing of an Agent Trained to Mimic</i>	
<i>LIPO-Generated Partners</i>	<i>15</i>
<i>Figure 7, Success with Specialists vs Training Method. Success rates are shown for each team, and the standard deviations across three test runs in parentheses.</i>	<i>18</i>

Abstract:

This study aims to improve human-Artificial Intelligence (AI) ad-hoc cooperation. Building upon Charakorn et al., 2023, this study introduces imitation learning as a possible alternative to traditional reinforcement learning (RL) for training cooperative agents. This study focuses on perfect information, full observability environments, specifically the Multi-Recipe Overcooked Environment as introduced in Wu et al. and expanded upon in Charakorn et al., 2023. By imitating random agents produced using the Learning Incompatible Strategies (LIPO) method as detailed in Charakorn et al., 2023, as opposed to simply training with them, imitation learning is tested as an alternative to traditional reinforcement learning. Although imitation learning was found to be less effective than traditional RL, this study offers possible insights to the impact the layout of an environment can have on the nature of cooperation, and how the complexity of the task and environment impact success rates. This research also adds to the body of knowledge about cooperative AI, a relatively new field (Bard et al.).

1. Introduction

Artificial Intelligence (AI) is a field of computer science which seeks to create programs that can mimic human intelligence (Sutton et al.). From emergency response vehicles (Zhao et al., 2022) to the medical field and financial decisions (Sutton et al.), AI is being used in a myriad of applications.

Ad-hoc human-AI cooperation remains a problem unsolved by AI (Bard et al., 2020). This means that when humans and AI algorithms need to cooperate, they often struggle (Bard et al., 2020). However, this form of cooperation is essential to the application of AI in the human-centric world (Bard et al., 2020). Human nurses and physicians need to cooperate with AI-controlled medical machines. Self-driving cars need to cooperate with human drivers. (Bard et al., 2020). As such, this paper proposes a new method for human-AI ad-hoc cooperation.

The remainder of Section 1 will describe previous methods used for human-AI ad-hoc cooperation. Section 1.1 will describe Reinforcement Learning, 1.2 will describe imitation learning, 1.3 will describe the Learning Incompatible Strategies (LIPO) method from Charakorn et al., 2023, and 1.4 will describe cooperative environments. Within 1.4, 1.4.1 will describe the popular Hanabi Learning Environment as described in Bard et al., and 1.4.2 will describe the Overcooked Environment as described in Carroll et al., as well as the environments based on the Overcooked Environment. 1.4.2.1 will describe the Multi-Recipe Overcooked environment as presented in Wu et al. and expanded upon in Charakorn et al., 2023, and 1.4.2.2 will describe the procedurally-generated Overcooked-inspired environments as introduced in Fontaine et al. 1.5 will state the goals and purpose of this study.

Section 2 will describe a new method for human-AI ad-hoc cooperation, and how the model was trained and tested. Section 3 will present the results of the method, and Sections 4 and 5 will conclude the findings of the research and discuss future work, respectively.

1.1 Reinforcement Learning Overview

This paper, as well as many others, use Reinforcement Learning (RL) to create their algorithms. RL is a field of AI which seeks to create an agent, or a decision-making algorithm, to accomplish a goal (Sutton et al.). The performance of the agent in a run can be expressed as a number, where a better performance is a higher number, and a worse performance is a lower number (Sutton et al.). This number is called the reward (Sutton et al.). Thus, the goal of a reinforcement learning agent is to make decisions which maximize the reward (Sutton et al.).

The way the agent makes decisions is by using a set of rules called a policy (Sutton et al.). The agent is capable of taking observations, taking actions, and receiving rewards. After rewards are received, the policy is adjusted using a reinforcement learning algorithm to maximize the reward in the future, should the same observation, or a similar observation, be seen in the future (Sutton et al.). The process is seen below in Figure 1.

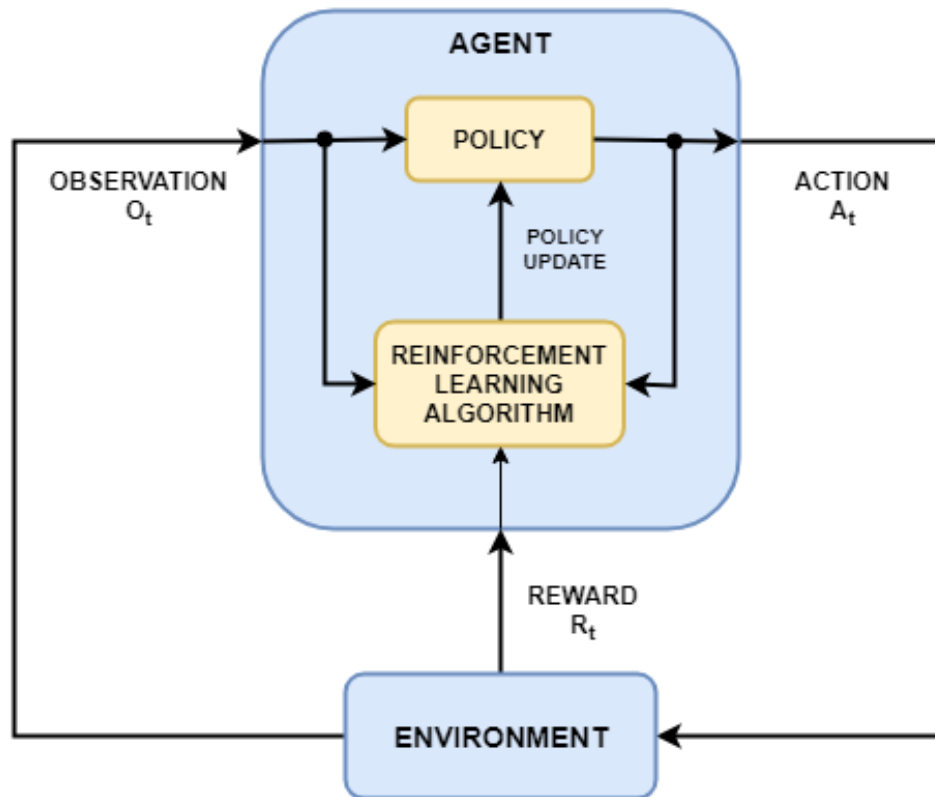


Figure 1 displays a simplified version of the reinforcement learning cycle. The agent is shown as both the policy and the reinforcement learning algorithm. The policy is capable of taking inputted observations, O_t about the state of the environment, and returning an action, A_t , which is the decision made based on the observation. The reinforcement learning algorithm is capable of receiving a reward, R_t , and updating the policy based on the reward to maximize R_t in the future (DeepMind, 1).

The final part of the reinforcement learning cycle is the environment. The environment is the space the agent exists in. It takes A_t as an input, and based on the environment's state, returns O_t to the policy and R_t to the algorithm. A_t changes the state of the environment, or the condition of the environment, which determines O_t and R_t (DeepMind, 1). Actions change the state of the environment (DeepMind, 1)..

One cycle takes one timestep, or t . In this timestep, the agent takes an action which may manipulate the state, then receives a reward and observation, which are used to change the policy and make the next decision. There are many timesteps in a single run, and the sum of the rewards accumulated throughout the run are the total rewards for the run. This total reward is used in other applications and methods (DeepMind, 1).

1.2 Imitation Learning Overview

Imitation Learning is a subsection of Reinforcement Learning and Supervised Learning. Supervised Learning is a field of AI which uses pre-generated data to train agents (Sutton et al.). Imitation Learning seeks to create an agent which is able to imitate the actions of another decision-making entity, usually a human. This other entity which the agent is trained to imitate is called the expert. In short, the expert generates a series of observation-action pairs which are used to train the agent. For each observation, the agent is given a reward according to how close its action was to the expert's action. In this way, the goal of the Reinforcement Learning agent is set to be to mimic, or imitate the expert's actions after each observation. (Codevilla et. al)

1.3 Learning Incompatible Strategies (LIPO) Overview

Learning Incompatible Strategies (LIPO) is a method from Charakorn et al., 2023 which seeks to create a set of diverse training partners for Reinforcement Learning training. Charakorn et al., 2023 posits that strategies which are incompatible are different. As such, Charakorn et al., 2023 devises a method which creates agents which are incompatible, and therefore different, to make a diverse population of random agents. These agents are later used as training partners for the RL Generalist agent, giving the Generalist exposure to many diverse strategies it may need to cooperate with. This helps to alleviate the issue of agents being inflexible to human suboptimality, as humans often take suboptimal, inefficient actions which are not taken by

automated training partners. As the Generalist trains with the semi-random partners, it learns about that partner’s sub-optimal policy, and is able to perform better if it encounters that strategy later (Charakorn et al., 2023).

1.4 Environments

Agents are often trained in virtual environments modelled after games (Bard, Foerster et al.). Chess and Poker were used to train adversarial AI, however, cooperative environments are often newer and not modelled after such popular games (Bard, Foerster et al.).

1.4.1 The Hanabi Learning Environment

One of the first cooperative environments was the Hanabi Learning Environment, introduced in Bard, Foerster et al., which was modelled after the card game Hanabi. Hanabi is a unique environment in that not only does it have cooperative elements, but also has imperfect information. This means that each player is not given full state information, so the observation of each agent at time t , O_t , differs from the state of the environment at time t , S_t . Since $O_t \neq S_t$, each agent must give and receive information to its teammates implicitly. In Hanabi, information about certain cards can be given sparsely by using in-game resources, however the act of giving a hint about a card can imply other information, such as whether or not the card is playable and therefore whether the card should or should not be played. Bard, Foerster et al. analyzes common strategies for the imperfect information challenges Hanabi proposes (Bard, Foerster et al.). While this paper does not use the Hanabi Learning Environment, it is important to recognize that the issue of human-AI ad-hoc cooperation is a separate challenge from imperfect information, even though the two are often connected. This paper focuses on solving exclusively the issue of human-AI ad-hoc cooperation in perfect information settings.

1.4.2 The Overcooked Environments

Another popular cooperative environment is the Overcooked Environment, introduced in Carroll et al., which was modeled after the popular video game *Overcooked*. Figure 2 below shows a graphic of the Overcooked Environment.

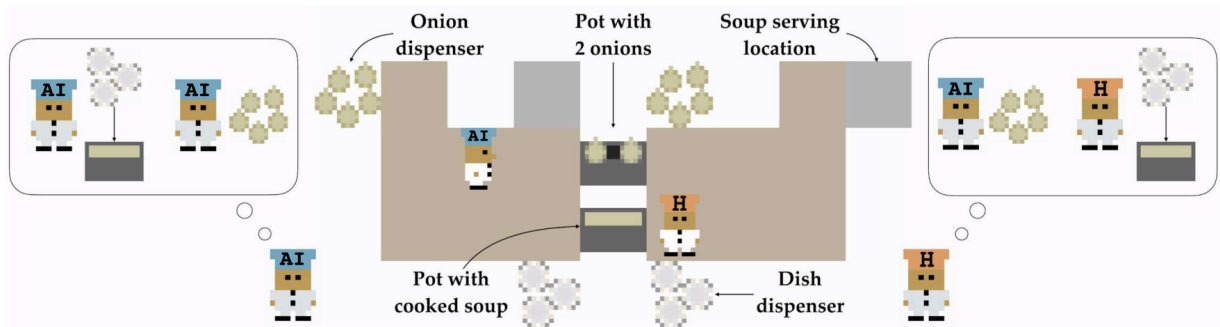


Figure 2, *The Overcooked Environment* (Carroll et al.)

The Overcooked Environment features two players, usually a human or human-proxy with an agent, who cooperate. The goal of the agent and the human are to score points in a certain amount of time by delivering soup items to the soup serving location within the environment. To do so, the agent and human must take onion items from the onion dispenser location and place them in pot locations. After a certain amount of time, the pot's state changes from holding onions to cooked soup, which can be interacted with a dish item to create a finished soup dish. This dish can be placed in the soup serving location to earn points (Carroll et al.).

This environment is deceptively simple. Agents who undergo self-training often form stubborn behaviors and patterns depending on the layout they were trained on. This leads to inflexibility when performing with human partners, who often make unpredictable, suboptimal choices, leading to poor cooperation (Carroll et al.).

The environment's layout heavily influences the most optimal strategy, and how complex coordination must be to succeed (Carroll et al.). The layouts from Carroll et al. are shown below in Figure 3.

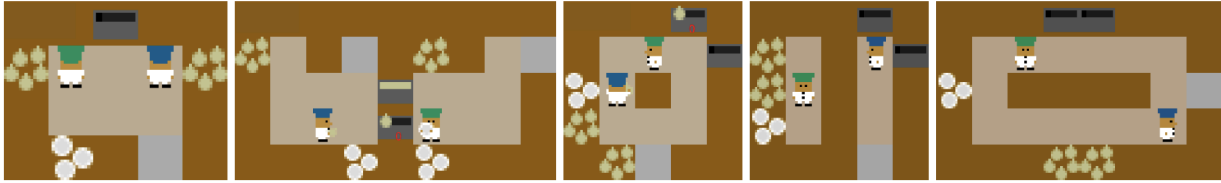


Figure 3, Experimental Overcooked Layouts (Carroll et al.)

While Carroll et al. delves further into depth into the differences in each layout and the issues they present, it is evident that an environment such as the layout to the far left is less complex than the layout directly to the right of it (Carroll et al.).

It should be noted that the Overcooked Environment, and the environments modelled after it, which are covered in the remainder of 1.4.2, are perfect information environments, where $O_t = S_t$ (Carroll et al.). This makes it different than Hanabi, which was an imperfect information environment, where $O_t \neq S_t$ (Bard et al.).

1.4.2.1 The Multi-Recipe Overcooked Environment

An extension to the Overcooked Environment is the Multi-Recipe Overcooked Environment as presented in Wu et al., which was extended in Charakorn et al., 2023, which this paper uses. This Multi-Recipe Overcooked Environment, as the name states, features multiple possible recipes that can be made simultaneously from the same ingredients. It proves to be a much more complex environment than the original Overcooked Environment for this reason. Figure 4 displays an overview of the Multi-Recipe Overcooked Environment as used in Charakorn et al., 2023, including an example of an initial state and an overview of all possible recipes.

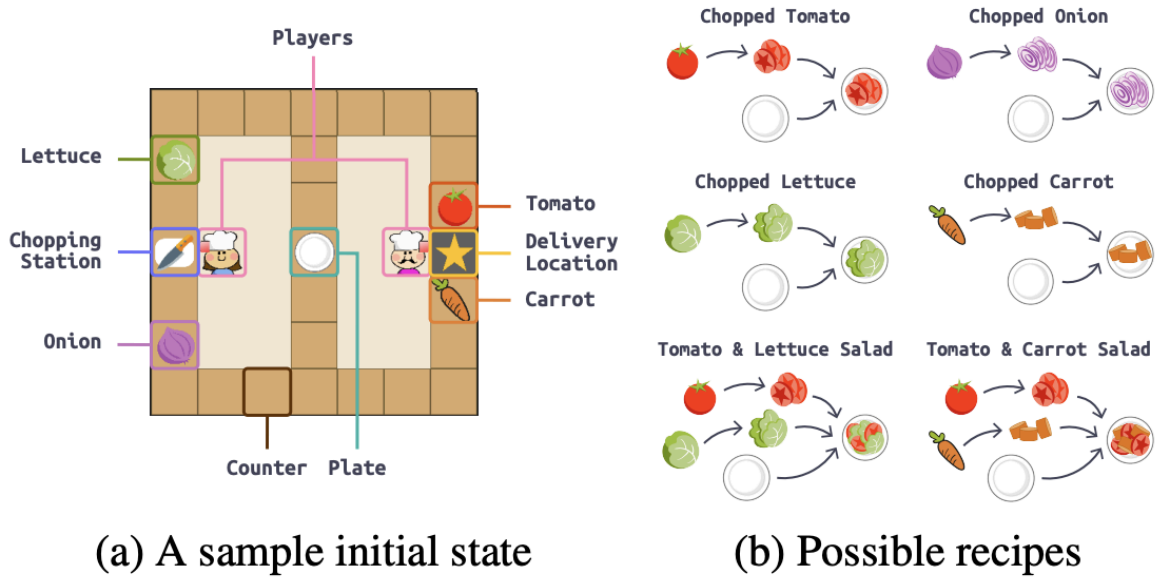


Figure 4, Overview of the Multi-Recipe Overcooked Environment (Charakorn et al., 2023)

It should be noted that Tomato and Carrot strategies are harder than those without these ingredients, since there is an additional step of passing the ingredients over the counter (Charakorn et al., 2023).

1.4.2.2 Procedurally Generated Overcooked Environments

In real applications, agents should not only be able to cooperate with a diverse pool of agents, but also within a diverse pool of environments. As such, Fontaine et al. introduces a method of generating new layouts for Single-Recipe Overcooked-inspired environments. These randomly generated environments are similar to human-designed ones, solvable, and are diverse (Fontaine et al.). While not used in this study, it is important to make the distinction between flexibility with respect to environments and flexibility with respect to cooperative partners. This paper focuses only on improving flexibility with respect to cooperative partners.

1.5 Statement of Purpose

This study seeks to improve upon ad-hoc human-AI cooperation in more complex settings, where ad-hoc human-AI teams perform worse than human-human teams (Charakorn et al., 2023). To do so, this study proposes a new method for training RL agents for use in the Multi-Recipe Overcooked Environment as expanded upon in Charakorn et al., 2023. This new method is an expansion of the LIPO method used in Charakorn et al., 2023, and uses imitation learning tools to add to the goal of the agent. It was hypothesized that by using imitation learning in addition to traditional RL methods in setting the reward function of the agent, the agent would better generalize the policies of the LIPO agents. Thus, this newly trained agent should be able to better cooperate with more diverse partners, leading to better performance with both human-proxy models and real humans. This ability to cooperate with multiple partners is called robustness, and this study seeks to improve the robustness of cooperative agents.

By improving human-AI ad-hoc cooperation in the Multi-Recipe Overcooked Environment, a relatively complex environment, applications in industry can be improved. Since this agent is extremely similar to previous agents, implementation in industry should be easy, should the user wish to switch from a previous agent to this new agent.

2. Methods and Materials

Figures 5 and 6 below show the steps taken to train and evaluate the agent. In short, this study seeks to create an imitation-learning based agent which imitates LIPO-generated agents. LIPO-generated agents form in pairs, with each pair containing two agents which cooperate well together, but each agent not performing well when not in its original pair. It was hypothesized that since LIPO-generated agents cooperate well within their own pairs, they should be able to cooperate with other suboptimal strategies when they are encountered.

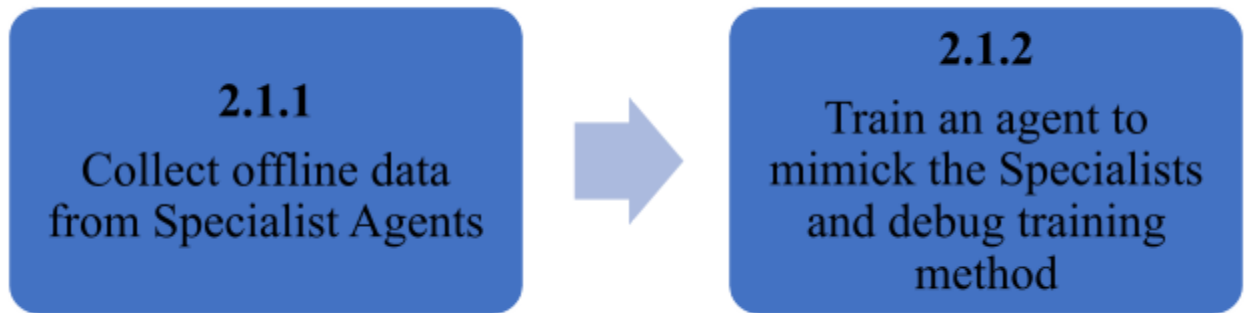


Figure 5, Overview of the Steps Taken to Create and Debug the Training Method

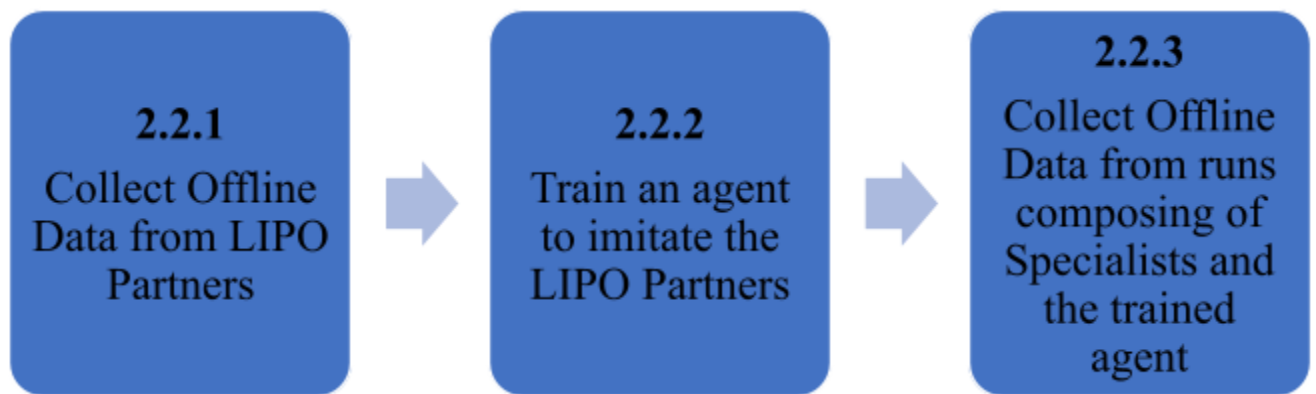


Figure 6, Overview of the Training and Testing of an Agent Trained to Mimic LIPO-Generated Partners

2.1 Creation of the Training Method

The first step was to create a training method that uses imitation learning to train an agent based on the actions of a set of previous runs. To do so, an agent was trained to mimic simple Specialist Agents.

2.1.1 Collection of Offline Data from Specialist Agents

Firstly, offline data was collected from Specialist Agents. These Specialist Agents, or Specialists, were simple agents trained in the Multi-Recipe Overcooked Environment to perform a single recipe. Thus, it had minimal robustness, making the policy of the agent extremely simple. These Specialists were provided by the mentor to the student scientist, as they had been used in previous projects by the mentor. To collect offline data, a Jupyter Notebook was used to allow the agents to simulate 7000 runs with each other, recording each action taken by each agent in each run. A notebook previously used by the mentor for a similar task was provided by the mentor, and the student scientist adapted this notebook to suit the needs of the project. This data was recorded in Python Pickle (.pkl) files.

2.1.2 Training an Agent to Mimic Specialists

After offline data was collected, an agent was trained to mimic the specialists. The student scientist added a new training method to a pre-existing agent class, according to specifications given by the mentor. The new training method used pyTorch Imitation Learning tools to create a simple supervised learning objective. In short, the agent was trained to mimic the actions of the specialists by setting the collective A_t of the Specialists to be the highest possible reward given the state S_t . The closer the action of the trained agent is to the Specialists' collective actions at each state, the higher the reward. In this way, the trained agent learns to mimic the actions and policies of the Specialists.

The purpose of this step was to verify that the training method ran without errors. While it was entirely possible to skip step 2.1, it was easier to create the training method with the Specialists due to the simplicity of the policies and the organization of the Specialists being extremely simple.

2.2 Training an Agent to Mimic LIPO-generated Training Partners

Step 2.2 uses the training method created in 2.1 to train a second agent. Step 2.2 is extremely similar to Step 1.1 in that both include gathering offline data of agents and using the training function. Truthfully, 2.2 is mostly applications of code written in 2.1 to fit the LIPO partners.

2.2.1 Collecting Offline Data from LIPO-Generated Partners

Step 2.2.1 is extremely similar to 2.1.1. The notebook from 2.1.1 was mildly changed to collect offline data from LIPO-Generated partners, which were diverse partners provided by the mentor to the student. Offline data was generated from the LIPO-generated partners for use in step 2.2.2.

2.2.2 Training an Agent to Mimic LIPO-Generated Partners

The same training method from step 2.1.2 was used with the offline data from step 2.2.1 to train an agent using imitation learning to mimic the behavior of LIPO-Generated Partners. An agent was created to be used in step 2.2.3.

2.2.3 Collecting Offline Data from the LIPO-Trained Agent

In this step, offline data of runs including the LIPO-Trained Agent created in step 2.2.2 and a specialist from step 2.1.1 was collected. This data is discussed in section 3. The notebook from steps 2.1.1 and 2.2.1 was copied and slightly modified to collect this data.

3. Results and Discussion

3.1 Results

In summary, this study's proposed imitation-learning based method was not better than existing RL methods. Figure 7 displays the display rates and standard deviations over 3 runs with Specialist Partners for Imitation from LIPO partners and RL from LIPO partners, as used in Charakorn et al., 2023.

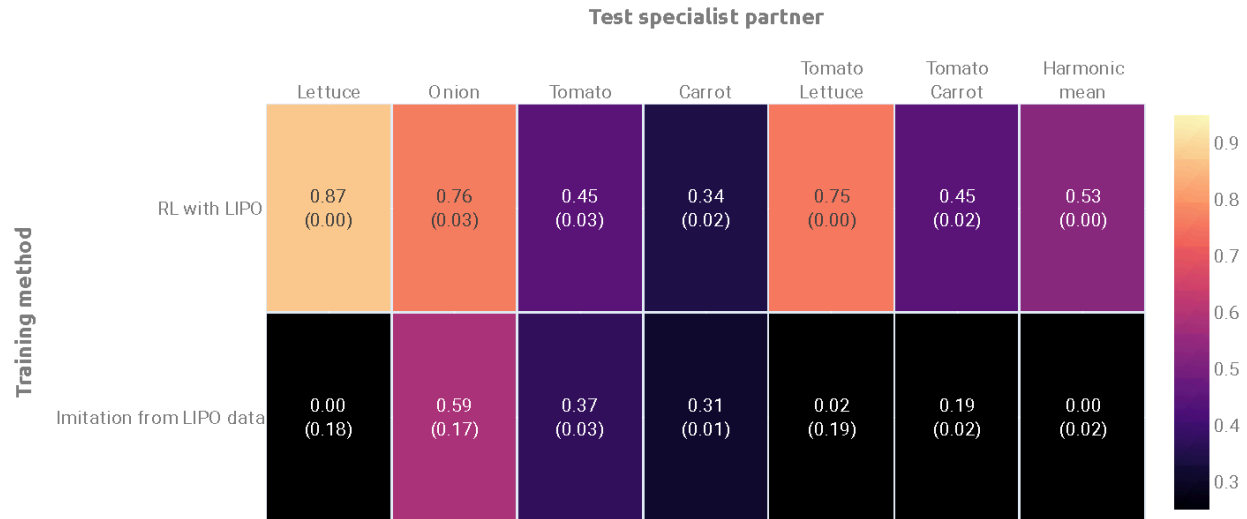


Figure 7, Success with Specialists vs Training Method. Success rates are shown for each team, and the standard deviations across three test runs in parentheses.

As seen above, Imitation Learning from LIPO data is worse in each possible team than RL with LIPO.

3.2 Discussion

While the results of this study were negative, there are still many valuable and interesting observations that can be taken from these findings.

3.2.1 High Standard Deviations for Imitation-Lettuce, Imitation-Onion, and Imitation-Tomato Lettuce Teams

It should be noted that the high standard deviation for the Lettuce, Onion, and Tomato Lettuce Specialists with Imitation from LIPO data are likely the cause of the roles of the partners shifting. In the Multi-Recipe Overcooked Environment, one agent gets access to Lettuce, Onion, and the chopping station (Chop Side Partner), while the other gets access to Tomato, Carrot, and the serving station (Serve Side Partner).

3.2.1.1 High Standard Deviation for Imitation-Lettuce and Imitation-Onion Teams

When a Lettuce or Onion Specialist is on the Serve Side, their policy states to move chopped Lettuce or Onion to the serving station. However, if neither ingredient is passed by the Chop Side Partner, there is incredibly poor performance. However, should the Specialist be placed on the Chop Side, their policy states to pass over their chopped preferred ingredient to the Serve Side Partner. Since it is always advantageous, though not optimal, to pass over a finished recipe, the Imitation Agent was likely able to achieve some points when it was on the serve side, since it was able to score using the Onion or Lettuce passed over. Which partner is on which side could have possibly impacted the performance of the team on that run, leading to a higher standard deviation between runs.

3.2.1.2 High Standard Deviation for Imitation-Tomato Lettuce Teams

A similar logic can be used to hypothesize as to the reasoning behind the high standard deviation for the Tomato Lettuce-Imitation team. A Tomato Lettuce Specialist on the Serve Side passes over Tomato to be chopped but will not serve recipes other than Tomato Lettuce Salads. If the Imitation agent on the Chop Side does not pass over Tomato Lettuce Salads, instead attempting to create Onion-based items, or anything but Tomato Lettuce Salads, no points will be scored. However, if the Tomato Lettuce Specialist is on the Chop Side, it will pass over Chopped Lettuce, and possibly create Tomato Lettuce Salads if it is presented with Tomato. Since it is guaranteed that the Imitation agent on the Serve Side will receive a servable item, the team will score points. Again, which partner is on which side could have possibly impacted the performance of the team on that run, leading to a higher standard deviation between runs.

3.2.2 Extremely Low Relative Performance with Imitation-Lettuce and Imitation-Tomato Lettuce Teams

3.2.2.1 Extremely Low Relative Performance with Imitation-Tomato Lettuce Teams

Tomato Lettuce Salads are naturally the most complex recipe due to the nature of the layout of the Multi-Recipe Overcooked Environment. They require Tomato to be passed over, both Tomato and Lettuce to be chopped, the two to be combined, and then for the dish to be passed over once again. It is also the only recipe which requires an ingredient from each side. Further research should be conducted to determine how agents interact in such complex cases.

3.2.2.2 Extremely Low Relative Performance with Imitation-Lettuce Teams

Chopped lettuce is one of the simplest recipes in the environment, equal in complexity to Chopped onion. However, Chopped Lettuce is the only Chop Side ingredient which is both a standalone recipe and an ingredient to a complex recipe (Tomato and Lettuce Salad). The exact interactions are unknown, and further research should be conducted as to how the layout and complexity of the environment shapes interactions.

3.2.3 Overall Lower Performance of Imitation Agent vs. RL Agent

Imitation Learning in such a simple form often performs poorly compared to RL due to issues with diversity among the LIPO-generated offline data. LIPO partners inherently work together well but are specialized within their pairs to do so. As such, the Imitation Agent likely learned policies that are too hyperspecialized to work with other LIPO policies to be applicable to the specialists. Because LIPO policies always work together well, they do not have to be robust strategies, as they are purposefully incompatible with other pairs. As such the policies learned by the Imitation Agent were likely suboptimal compared to the policies learned through traditional RL training, since the RL agent had to learn to cooperate with each LIPO policy independently, while the Imitation Agent was only trained on agents within their pairs

4. Conclusions

This paper attempted to improve ad-hoc human-AI cooperation. It attempts to use imitation learning as a viable alternative to traditional RL for these purposes and tests the two methods in the Multi-Recipe Overcooked Environment. The Imitation Learning agent did not do as well as the traditional RL agent when paired with Specialists, who had suboptimal, non-robust strategies which are similar to possible human strategies.

5. Future Work

In the future, implementing Multi-Recipe Overcooked Environments inspired by Wu et al. and Charakorn et al., 2023 with procedurally generated layouts such as in Fontaine et al. is interesting. Since the layout of the environment impacted the results of certain teams in this paper, training and testing on a variety of layouts may give further insight into how best to train agents. Mapping interactions using methods similar to the ones introduced in Nalepka et al. may provide further insight into how agents interact and delegate roles to unknown partners in perfect information settings.

In addition, using a more complex Imitation Learning algorithm, such as the DAgger algorithm introduced in Ross et al. may prove more successful than this extremely simple imitation learning algorithm. While more computationally expensive, it may prove more effective than current RL methods.

6. References

Bard, N., Nicholas, D., Szepesvari, C., & Bowling, M. (2015). Decision-Theoretic Clustering of Strategies. In AAAI Workshop: Computer Poker and Imperfect Information.

Bard, Nolan, Foerster, J. N., Chandar, S., Burch, N., Lanctot, M., Song, H. F., ... Bowling, M. (2020). The Hanabi challenge: A new frontier for AI research. *Artificial Intelligence*, 280(103216), 103216. doi:10.1016/j.artint.2019.103216

Bowling, M., Burch, N., Johanson, M., & Tammelin, O. (2017). Heads-up limit hold'em poker is solved. *Communications of the ACM*, 60(11), 81–88. doi:10.1145/3131284

Carroll, M., Shah, R., Ho, M. K., Griffiths, T. L., Seshia, S. A., Abbeel, P., & Dragan, A. (2019). On the utility of learning about humans for human-AI coordination. Retrieved from <http://arxiv.org/abs/1910.05789>

Charakorn, R., Manoonpong, P., & Dilokthanakul, N. (2020). Investigating partner diversification methods in cooperative multi-agent deep reinforcement learning. In *Communications in Computer and Information Science. Communications in Computer and Information Science* (pp. 395–402). doi:10.1007/978-3-030-63823-8_46

Charakorn, R., Manoonpong, P., & Dilokthanakul, N. (2022). Generating diverse cooperative agents by learning incompatible policies. Retrieved from <https://openreview.net/pdf?id=a7vLnGKGIjY>

DeepMind, G. [@Google_DeepMind]. (2015a, May 13). RL course by David Silver - Lecture 1: Introduction to Reinforcement Learning. Retrieved 11 January 2024, from https://www.youtube.com/watch?v=2pWv7GOvuf0&list=PLqYmG7hTraZBiG_XpjnPrSNw-1XQaM_gB&index=4

Fontaine, M. C., Hsu, Y.-C., Zhang, Y., Tjanaka, B., & Nikolaidis, S. (2021). On the importance of environments in human-robot coordination. Retrieved from <http://arxiv.org/abs/2106.10853>

Galatzer-Levy, I. R., Ruggles, K., & Chen, Z. (2018). Data science in the Research Domain Criteria era: Relevance of machine learning to the study of stress pathology, recovery, and resilience. *Chronic Stress* (Thousand Oaks, Calif.), 2, 247054701774755. doi:10.1177/2470547017747553

Liu, S., Cao, J., Wang, Y., Chen, W., & Liu, Y. (2021). Self-play reinforcement learning with comprehensive critic in computer games. *Neurocomputing*, 449, 207–213. doi:10.1016/j.neucom.2021.04.006

Mariño, J. R. H., & Toledo, C. F. M. (2022). Evolving interpretable strategies for zero-sum games. *Applied Soft Computing*, 122(108860), 108860. doi:10.1016/j.asoc.2022.108860

Nalepka, P., Gregory-Dunsmore, J., & Simpson, J. (n.d.). UC Merced Proceedings of the Annual

Meeting of the Cognitive Science Society Title Interaction Flexibility in Artificial Agents
Teaming with Humans Permalink Publication Date. Retrieved January 12, 2024, from
<https://escholarship.org/content/qt9ks6n70q/qt9ks6n70q.pdf>

Rabinowitz, N. C., Perbet, F., Song, H. F., Zhang, C., Eslami, S. M. A., & Botvinick, M. (2018).
Machine Theory of mind. Retrieved from <http://arxiv.org/abs/1802.07740>

Ribeiro, J. G., Martinho, C., Sardinha, A., & Melo, F. S. (2022). Assisting unknown teammates
in unknown tasks: Ad hoc teamwork under partial observability. Retrieved from
<http://arxiv.org/abs/2201.03538>

Ross, S., Gordon, G., Bagnell, J., & Learning, M. (n.d.). A Reduction of Imitation Learning and
Structured Prediction to No-Regret Online Learning. Retrieved January 12, 2024, from
<https://arxiv.org/pdf/1011.0686.pdf?fbclid=IwAR0lZsurW9XmT4mXhBKVNNi2Gtgt6yi>
cS0HW_FvuD7-WnZvXkOH-E_z2K48

Shuai, H., Li, F., She, B., Wang, X., & Zhao, J. (2023). Post-storm repair crew dispatch for
distribution grid restoration using stochastic Monte Carlo tree search and deep neural
networks. *International Journal of Electrical Power & Energy Systems*, 144(108477),
108477. doi:10.1016/j.ijepes.2022.108477

Sutton, R. S., & Barto, A. G. (2012). *An Reinforcement Learning: Introduction*. Mit Press.
What Is Reinforcement Learning? - MATLAB & Simulink - MathWorks Deutschland. (n.d.).

Retrieved 11 January 2024, from <https://de.mathworks.com/help/reinforcement-learning/ug/what-is-reinforcement-learning.html>

Wu, S.A., Wang, R.E., Evans, J.A., Tenenbaum, J.B., Parkes, D.C. and Kleiman-Weiner, M. (2021), Too Many Cooks: Bayesian Inference for Coordinating Multi-Agent Collaboration. *Top. Cogn. Sci.*, 13: 414-432. <https://doi.org/10.1111/tops.12525>

Zhao, R., Song, J., Yuan, Y., Haifeng, H., Gao, Y., Wu, Y., ... Wei, Y. (2021). Maximum Entropy Population-Based Training for zero-shot human-AI coordination. Retrieved from <http://arxiv.org/abs/2112.11701>